Frequency-Based Replacement for DLRM Embedding Caches: A Trace-Driven Study and a Synthesizable LFU Controller

[Working title, to be confirmed]

[Author names to be confirmed]
[Affiliation to be confirmed]

Abstract*—Embedding lookups in deep-learning recommendation models (DLRMs) touch a few rows far more often than the rest, which makes caching them attractive, but which replacement policy to use is an open question. We compare seven policies by trace simulation on the first 2,000,000 and the first 10,000,000 rows of the 45,840,617-row Criteo Display Advertising Challenge training file, with either one cache per table or one cache shared by all 26 tables. With a 256-vector cache per table, both LFU variants beat LRU (LFU with per-address counters kept after eviction: 71.8034% vs. 67.3151% hits at 2M rows, 68.3280% vs. 67.0796% at 10M). With one shared 256-entry cache, LFU that resets an address's count on eviction beats LRU (47.6719% vs. 41.4010% at 2M, 48.1592% vs. 41.2640% at 10M), while the variant that keeps counts does not (41.0006%, 38.4896%). ARC and W-TinyLFU have the highest online hit rates at the smallest capacities in both organisations, at the price of extra state. GPU timing of 26 synthetic EmbeddingBag tables on one H100 and one H200 used CUDA events only (no hardware counters) and is launch-bound at small batches. We also build a synthesizable LFU cache controller whose default counter mode belongs to the reset-on-eviction family. On the NanGate45 open cell library (not TSMC; pre-layout; area as the sum of cell areas; no power analysis) it reaches 636.1, 603.0 and 556.6 MHz at depths 16, 64 and 256 (counter mode 0), within a synthesis-flow noise floor of 4.92–16.23%. Zero-delay gate-level simulation with cell models generated from the liberty file matches the RTL in every one of 200,000 random cycles for all 12 netlists; formal equivalence is proven at depth 16 only.

I. Introduction

Deep-learning recommendation models combine dense features with many categorical features, each mapped to a row of an embedding table [1]. Characterisation of production recommendation models has shown that these models place different demands on hardware than vision or language networks [2], and a line of systems work exploits the skew of embedding accesses: Bandana keeps embeddings in non-volatile memory with a DRAM cache [3], FAE keeps frequently accessed ("hot") embeddings in GPU memory [4], ScratchPipe builds a GPU-side embedding cache that looks at future accesses [5], and RecNMP moves embedding operations near memory [6]. Each of these needs a rule for which rows to keep. Skew suggests frequency-based replacement (LFU) rather than recency (LRU), but LFU comes in variants that behave differently, and the organisation of the cache (one per table or one for all tables) changes the competition between tables.

This paper reports a trace-driven comparison and a hardware building block, and states the limits of each next to the result:

- A comparison of LRU, two LFU variants, TinyLFU with exact counters, W-TinyLFU, ARC and Belady's offline bound on Criteo traces, for per-table and shared caches, at two trace lengths (Sections III, V).
- The finding that the ranking of the two LFU variants depends on the cache organisation, and that ARC and W-TinyLFU are the strongest online policies at small capacities (Section V).
- GPU timing of embedding lookups without hardware counters, with the launch-bound region identified (Section VI).
- A synthesizable LFU controller, its timing and area on NanGate45 across design iterations with the flow noise floor, and its verification by mutation testing, gate-level simulation and formal equivalence (Sections IV, VII).

Every number in this paper is generated from committed result files (`results/P7/TABLES.md`); each table and figure names its source.

II. Background

LRU evicts the least recently used entry. **LFU** evicts the entry with the fewest accesses; variants differ in whether an address's count survives its eviction. We call the variant that discards the count on eviction and restarts it at one on re-insertion *LFU, reset on eviction*, and the variant that keeps a per-address count in an unbounded table *LFU, persistent counters*; the latter is an idealised model of history-keeping LFU. **TinyLFU** adds an admission filter that compares the frequency of a new item with that of the eviction candidate; **W-TinyLFU** places a small LRU window in front of the main cache [7] and estimates frequencies with a Count-Min sketch [8]. **ARC** balances a recency list and a frequency list and keeps ghost lists of recently evicted keys to adapt the balance [9]. **Belady's** MIN policy evicts the entry used furthest in the future; it needs the whole trace and serves as an upper bound [10].

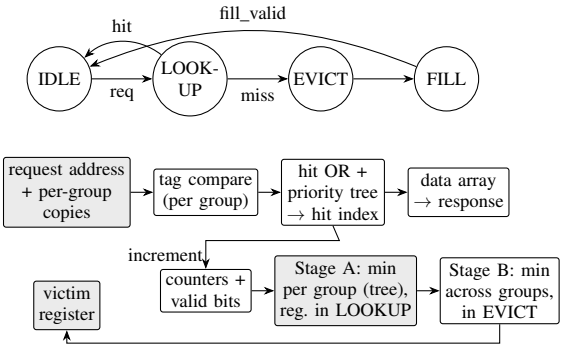


Fig. 1. The LFU controller (`rtl/v2/lfu_cache_controller.v`, tag `rtl-v2-final`): FSM (top) and datapath (bottom; shaded blocks are registers). A hit returns to IDLE; a miss selects the victim (Stage A registered in LOOKUP, Stage B in EVICT) and installs the fill in FILL. Drawn from the RTL source.

III. Trace Simulation Setup

Data. We use the Criteo Display Advertising Challenge training file (Kaggle, 2014) [11], 45,840,617 rows with 26 categorical columns. Each column is one embedding table and each distinct hashed value one vector. We replay the first 2,000,000 and the first 10,000,000 rows, i.e. a prefix of the file; nothing is claimed for the full file. Empty fields produce no access. Rank-frequency fits of a Zipf exponent per table give α from 1.0733 to 4.9724 at 2M rows and 1.0759 to 5.6033 at 10M rows (24 of 26 tables; two tables have only three distinct values).

Cache organisations. *Per-table*: each table has its own cache of C vectors (26 caches; C from 64 to 100,000). *Shared*: one cache of C entries holds vectors of all tables (key = table and value; C = 256, 1,000, 10,000 and 100,000), and the accesses of one row are issued in table order 0 to 25. The shared ordering is a modelling choice. Capacities are in vectors, not bytes.

Policies. The seven policies of the previous section. TinyLFU is run with exact counters and a 1% window; W-TinyLFU with a Count-Min sketch and a 1% window. The LFU variants evict the smallest count and break ties in favour of the entry that entered the cache earliest; their counts saturate at 255. The simulators are covered by unit tests and by mutation tests that check the tests fail on deliberately broken policies.

IV. LFU Controller

Structure. The controller (Fig. 1) stores tags, 128-bit vectors, frequency counters and valid bits in flip-flops (no SRAM macro). Parameters: address width 20, counter width 6 (saturating at 63), 16 groups for victim selection, and a depth of 16, 64 or 256 lines. On a request the FSM goes IDLE→LOOKUP; the tag compare runs per group on registered copies of the request address, and a balanced priority tree gives the lowest matching line. On a hit the counter is incremented and the response is returned. On a miss, Stage A (a reduction tree per group, registered in LOOKUP) and Stage B (a tree across groups, in EVICT) find the line with the smallest count; an invalid line counts as smaller than any valid line. FILL installs the returned vector.

Counter modes. With `COUNTER_MODE` 0 (the default) the count of a newly installed tag is set to one, so an address's history is lost on eviction: the mode corresponds to the reset-on-eviction family of Section III, but it is not the simulated policy. With mode 1 the new tag continues from the count left in the slot. Neither mode keeps a per-address history, so the persistent-counter LFU of the simulations has no RTL counterpart.

Differences from the simulated policy. We compared the tie-break rules in the source code. Among lines with the smallest count, the RTL evicts the one with the lowest slot index (both reduction trees keep the left operand on ties), whereas the simulator evicts the entry that entered the cache earliest. Because a new tag takes the slot of the evicted line, slot index and age differ after the first replacement, so the two can choose different victims. The counters also saturate at different values (63 in the RTL, 255 in the simulator). The RTL was not replayed on the Criteo traces, so the hit rates of Section V are not the RTL's.

Synthesis flow. Yosys 0.33 with ABC [12], [13] maps the RTL to the NanGate45 open cell library (typical corner) [14]; OpenSTA [15] reports setup and hold slack with the library's wire-load model and an ideal clock. NanGate45 is an open 45 nm library, not a TSMC process; all results are pre-layout, area is the sum of the cell areas, and no power analysis was run. Each configuration (depth 16, 64, 256; mode 0, 1) is synthesized for a 1 GHz and a 500 MHz target, and Fmax $= 1/(T - \mathrm{WNS})$ of the worst setup path. The current flow passes the target to ABC through a delay-oriented script whose target is the clock period minus 141.5 ps (clock-to-Q plus setup on the worst path of an earlier depth-16, mode-0 run). The script was chosen on that single depth-16 pilot, and no ablation of its parts was run. To measure the noise of the flow itself we synthesize 8 designs with the same logic (the final RTL, a flat-vector form of it, and three one-line rewrites of each) and report the spread of their Fmax.

Verification. A self-checking testbench with 7 tests runs on the RTL for every depth and mode, and a lockstep testbench compares each RTL version with earlier versions on random traffic. Mutation testing checks that the tests fail on broken copies of the RTL. For the netlists we run a functional gate-level simulation with zero delay in Icarus Verilog [16]: the cell models are generated by Yosys from the liberty file and checked against its cell functions, and no SDF timing is used. Yosys equivalence checking (`equiv_make`, `equiv_induct`) compares RTL and netlist.

V. Results: Replacement Policies

Per-table caches. With 256 vectors per table (Table I), both LFU variants beat LRU at both trace lengths, and on all tables combined they beat LRU at every capacity from 64 to 100,000 vectors (`results/P3/ANALYSIS.md`, Section 3). The margin of the persistent-counter variant shrinks with the longer trace (71.8034% at 2M, 68.3280% at 10M), and per table it falls below LRU on six tables at 10M rows (Fig. 2). TinyLFU with exact counters is below LRU at 10M rows

TABLE I

HIT RATE (%) WITH 256-VECTOR CACHES. PER-TABLE: ONE CACHE OF 256 VECTORS PER TABLE (26 CACHES); SHARED: ONE CACHE OF 256 ENTRIES FOR ALL TABLES. FIRST 2M AND 10M ROWS OF CRITEO DAC; TRACE SIMULATION.

	Per-table		Shared	
Policy	2M	10M	2M	10M
LRU	67.3151	67.0796	41.4010	41.2640
LFU, reset on eviction	70.0951	70.3276	47.6719	48.1592
LFU, persistent counters	71.8034	68.3280	41.0006	38.4896
TinyLFU, exact counters	69.7367	64.5267	–	–
W-TinyLFU (Count-Min)	73.2679	73.1009	50.9749	50.9101
ARC	72.6030	72.4225	50.9225	50.8773
Belady (offline bound)	78.6721	78.4180	–	–

Source: `results/P7/TABLES.md` (P3, P3b), generated from `results/P3/rows_*/figs/hit_rates.csv` (commit a84d9a9) and `results/P3b/rows_*/parts/*.csv` (commits c81407a, 79f9adc). –: not run in the shared-cache simulator.

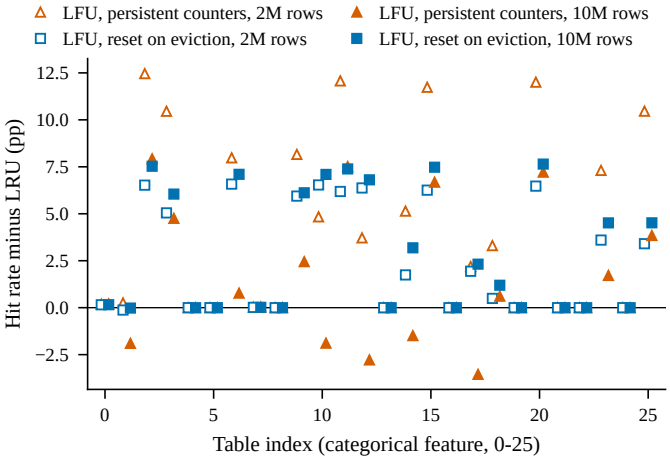


Fig. 2. Per-table caches of 256 vectors: hit rate of each LFU variant minus that of LRU, per table, at 2M (open) and 10M rows (filled); zero means the same hit rate as LRU on that table. Source: `results/P3/rows_*/figs/hit_rates.csv`, commit a84d9a9; script `paper/v2/figs/fig_pertable.py`.

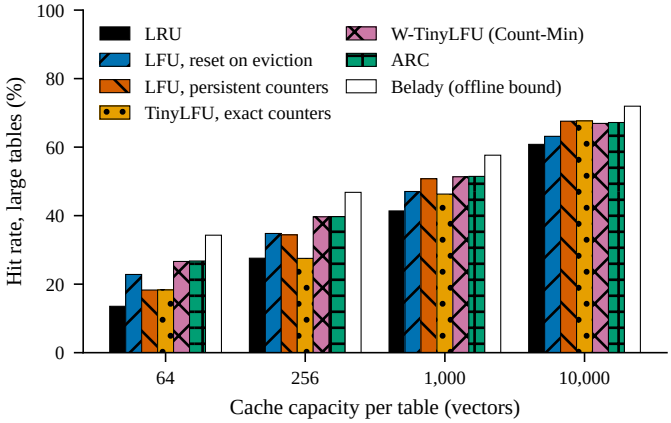


Fig. 3. Per-table caches, the five large tables (2, 3, 11, 15, 20) combined: hit rate by policy and capacity, 10M rows. Source: `results/P3/rows_10000000/figs/hit_rates.csv` and `zipf_per_table.csv` (commit a84d9a9); script `paper/v2/figs/fig_large.py`.

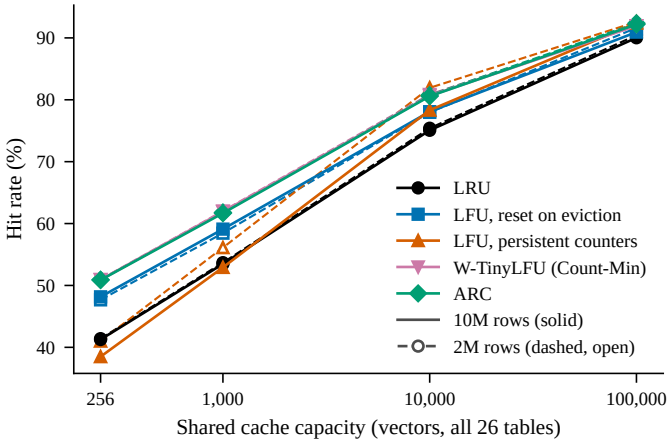


Fig. 4. One cache shared by all 26 tables: total hit rate vs. capacity; 10M rows solid, 2M rows dashed with open markers. Source: `results/P3b/rows_*/parts/*.csv` (commits c81407a, 79f9adc); script `paper/v2/figs/fig_shared.py`.

(64.5267% vs. 67.0796%). W-TinyLFU and ARC are the best online policies at 256 vectors; Belady's bound is 78.6721% and 78.4180%.

Five tables (2, 3, 11, 15 and 20) have at least 150,000 distinct values at both trace lengths. Fig. 3 shows them combined: at 64 vectors per table, reset-on-eviction LFU, W-TinyLFU and ARC are well above LRU, and Belady's bound stays above every online policy at every capacity shown.

Shared cache. With one cache for all tables the picture changes (Fig. 4). At 256 entries, reset-on-eviction LFU beats LRU at both trace lengths (47.6719% vs. 41.4010% at 2M, 48.1592% vs. 41.2640% at 10M), and it does so at every capacity tested. The persistent-counter variant is below LRU at 256 entries at both lengths (41.0006% and 38.4896%) and at 1,000 entries at 10M rows (52.9099% vs. 53.4740%); it is above LRU at 10,000 and 100,000 entries, and at 100,000 entries it has the highest hit rate of all policies (92.5839% and 92.3924%). At 256 and 1,000 entries, W-TinyLFU and ARC are the best policies. The persistent-counter variant also loses the most from the shorter to the longer trace.

What the comparison shows. The two LFU variants swap places between the two organisations: with per-table caches the persistent-counter variant is ahead at 2M rows, with a shared cache it is behind LRU at small capacity while the reset variant is ahead. In a shared cache, the tables compete for the same entries, and the persistent variant keeps counts for every address it has seen; we did not isolate which of these causes the difference. ARC and W-TinyLFU are the strongest online policies at small capacities in both organisations, but they need extra state (ghost lists for ARC, a sketch and a window for W-TinyLFU); we did not measure that cost in hardware.

VI. RESULTS: GPU TIMING

Setup. A synthetic model with 26 equal EmbeddingBag tables in PyTorch [17] (sum pooling, one lookup per table and sample) followed by an MLP runs fp32 inference on one H100

TABLE II

FINAL CONTROLLER (TAG `RTL-V2-FINAL`) ON NANGATE45, TIMING-DRIVEN FLOW, PRE-LAYOUT. FMAX FROM THE 1 GHZ NETLISTS; THE 500 MHZ COLUMN IS THE NETLIST SYNTHESIZED FOR 500 MHZ; NOISE = FMAX SPREAD OF 8 SAME-LOGIC DESIGNS. AREA = SUM OF CELL AREAS; NO POWER ANALYSIS.

Depth	Mode	Cells	FFs	Area (μm^2)	Fmax	500 MHz	Noise
16	0	11,466	2,957	26744.438	636.1	met	16.23%
16	1	11,057	2,957	26626.866	659.5	met	7.71%
64	0	37,939	10,431	94385.578	603.0	met	6.24%
64	1	38,692	10,431	95100.852	611.2	met	4.92%
256	0	149,620	40,225	369529.860	556.6	−0.0357 ns	8.53%
256	1	150,254	40,225	370226.514	542.3	−0.0015 ns	7.02%

Fmax in MHz; FFs = flip-flops. 500 MHz: "met" or the worst negative slack of the 500 MHz netlist. At depth 256 the shortfall is far inside the noise, so meeting 500 MHz is not demonstrated. Source: `results/P7/TABLES.md` (P5e, noise floor), generated from `results/P5e/new/d*/sta.txt` and `stat.txt` (commit 37c353c) and the noise-floor designs.

TABLE III

VERIFICATION OF THE FINAL CONTROLLER. GLS = FUNCTIONAL GATE-LEVEL SIMULATION, ZERO DELAY, CELL MODELS GENERATED BY YOSYS FROM THE LIBERTY FILE.

Check	Result
Mutation (RTL)	113 mutant runs killed by the unit tests, 10 more by the lockstep check, 33 runs of equivalent mutants
GLS, unit tests	all 7 tests pass on the depth-16 and depth-64 netlists (Icarus 12) and on the depth-16 and depth-256 netlists (Icarus 11)
GLS, lockstep	12 netlists, 200,000 random cycles each: 0 mismatching cycles on all 5 outputs, no X
Equivalence	depth 16 only: 4 netlists, 2977 of 2977 equivalence points proven

Source: `results/P7/TABLES.md` (P5f, mutation), generated from `results/P5f_gls/` (commits d7a2b54, 18b6aeb, 215d493) and `results/P5d/mutation.txt` (tag rtl-v2-final).

and one H200 (script `tools/profile_embedding.py`). Table rows are 10^6 or 10^7, the dimension 64 or 128, the batch 64 to 8192, with uniform, Zipf and Criteo-hashed indices. Timing uses CUDA events only: the profiler `ncu` was not available on the H100 node and refused to collect counters on the H200 node (`ERR_NVGPUCTRPERM`), so there are no hardware counters, and nothing here shows memory-boundedness or DRAM traffic. On the H100, 40 of 160 configurations were skipped because the 10^7-row, dimension-128 tables did not fit in GPU memory.

Launch-bound region. We call a configuration launch-bound if its CPU wall time per iteration is within 5% of the batch-64 value for the same tables and index pattern. Up to batch 1024 on the H100 and 512 on the H200 every series in Fig. 5 is launch-bound: there the share is a share of kernel-launch time, not of lookup cost. At batch 8192 the embedding share is 0.4224–0.4553 on the H100 and 0.4024–0.4358 on the H200 over all configurations that ran. The model has 26 tables of equal size; production models have tables of very different sizes, so these shares are not a statement about any production model.

VII. RESULTS: CONTROLLER HARDWARE

Design iterations. Fig. 6 follows the controller through three RTL changes and one flow change. The baseline reached 203.1, 194.7 and 167.4 MHz at depths 16, 64 and 256 (mode 0). Replacing serial minimum and priority loops by balanced trees raised depth 256 to 451.2 MHz with the earlier flow; the timing-driven flow then gives 636.1, 603.0 and 556.6 MHz on the same RTL. The noise bands are wide: with the earlier flow the 8 same-logic designs spread by 1.96% to 16.62% in Fmax, with the current flow by 4.92% to 16.23%, so differences of that size between two designs are not evidence of a design effect.

Final controller. Table II lists the final design. The 500 MHz netlists meet 500 MHz at depths 16 and 64 in both modes. At depth 256 they miss by 0.0357 ns (mode 0, 491.2 MHz) and 0.0015 ns (mode 1, 499.6 MHz); these shortfalls are far inside the noise of 8.53% and 7.02%, so whether depth 256 meets 500 MHz is not demonstrated either way. Compared with the earlier flow on the same RTL, the timing-driven flow uses 1.52% and 1.94% more area at depth 64 and 2.81% and 2.01% more at depth 256, and at depth 64 it has more nets that violate the slew or capacitance limit (2 to 7 in mode 0, 3 to 6 in mode 1; the reset net, which drives every flip-flop before any buffer tree is built, is excluded from these counts). A layout flow would buffer such nets, and timing would change.

Verification. Table III summarises the checks. In the lockstep run the netlist and the RTL see the same random requests and fills; the hit flag, response valid, response data, fill request and fill address are compared every cycle. The gate-level simulation has no delays, so it checks the logic of the netlists, not their timing. The equivalence proof treats the asynchronous reset as synchronous and starts the induction from matched registers being equal; it completed at depth 16 only.

VIII. LIMITATIONS

- *Data*: only the first 2,000,000 and 10,000,000 rows of the 45,840,617-row file; results changed between the two lengths (most for the persistent-counter LFU), so longer traces may change them again.
- *Organisation*: per-table and shared results are not comparable one to one, and the shared order of accesses within a row is a modelling choice.
- *Simulator vs. RTL*: tie-break and counter width differ (Section IV); the RTL was not replayed on the traces; there is no RTL for persistent per-address counters, ARC or W-TinyLFU.
- *GPU timing*: CUDA events only, no hardware counters; a synthetic model with 26 equal tables and one lookup per table; one GPU of each type; small batches are launch-bound.
- *Hardware*: NanGate45, not TSMC; pre-layout with a wire-load model and ideal clock; area is a sum of cell areas; no power; Fmax differences below the noise floor are not meaningful; the timing-driven flow was tuned on a single depth-16 pilot without ablation, uses up to 2.81% more area, and adds slew/capacitance violations at depth 64.

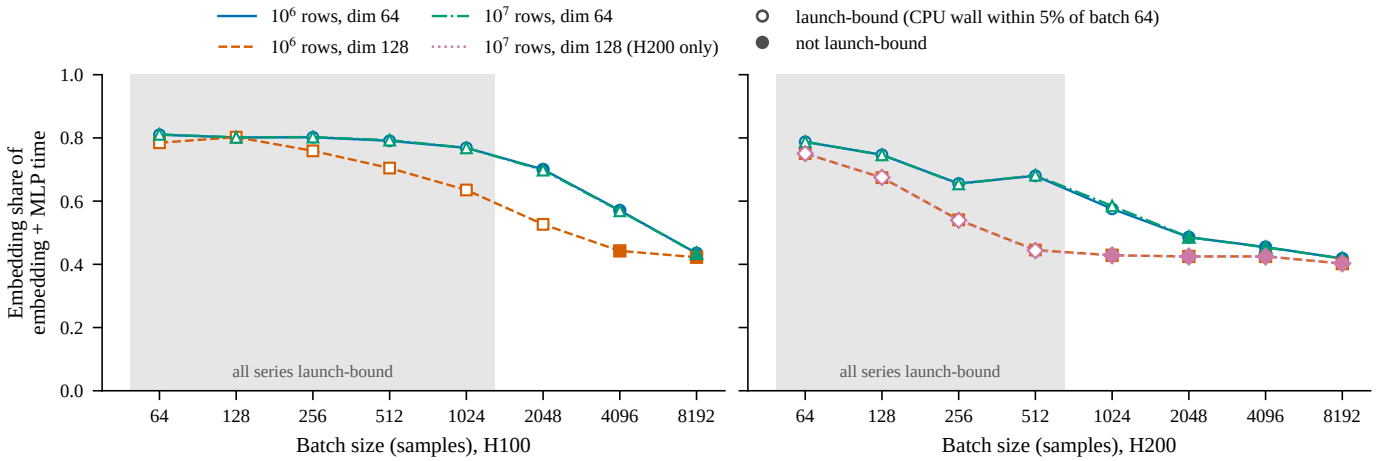


Fig. 5. Median embedding share of embedding + MLP time vs. batch size, Criteo-hashed indices, fp32 inference on one H100 (left) and one H200 (right); CUDA events only, no hardware counters. Open markers: launch-bound (CPU wall time per iteration within 5% of that at batch 64 for the same rows, dimension and index pattern). On the H100, 40 of 160 configurations were skipped for memory (10^7 rows, dimension 128). Source: `results/P4/h100_413621/results.csv`, `results/P4/h200_413622/results.csv` (commit daf1601); script `paper/v2/figs/fig_profile.py`.

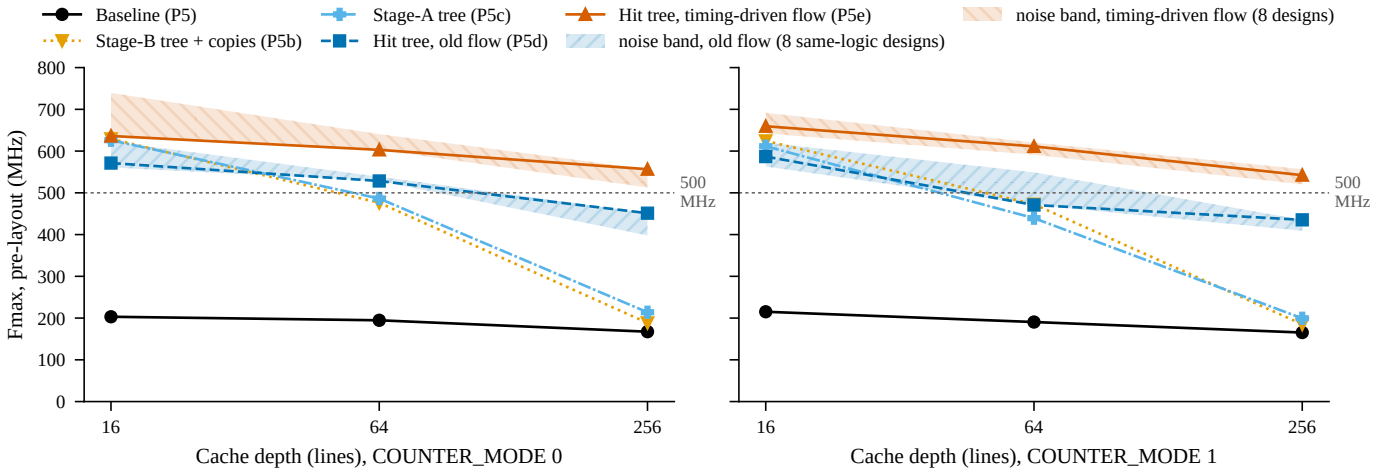


Fig. 6. Pre-layout Fmax on NanGate45 (not TSMC) of the 1 GHz netlists vs. depth for each design iteration: baseline, Stage-B reduction tree with address copies, Stage-A reduction tree, hit-index priority tree with the earlier flow, and the same RTL with the timing-driven flow. Shaded: min–max Fmax of 8 designs with identical logic (flow noise) for the two flows. Source: `results/P5*/d*_m*_1000MHz/sta.txt` and the noise-floor designs listed in `results/P7/TABLES.md`; final RTL tag `rtl-v2-final`, flow commit 37c353c; script `paper/v2/figs/fig_fmax.py`.

- *Verification*: gate-level simulation is zero-delay; equivalence is proven only at depth 16.
- *Earlier work*: an earlier gem5 study of this project (a CPU with DDR4, not a GPU [18]) is not reproduced here; the statistics of most of its runs are not available, and we use none of its numbers.

IX. Conclusion

On Criteo traces, frequency information helps embedding caches, but how it is kept matters, and the answer depends on whether tables share a cache. Per-table caches favour both LFU variants over LRU; a shared cache favours LFU that resets counts on eviction and penalises the variant that keeps them, at small capacities. ARC and W-TinyLFU are the strongest online policies at small capacities, at a hardware cost we have not measured. The synthesizable controller, whose default mode belongs to the reset-on-eviction family, reaches 542.3–659.5 MHz pre-layout on NanGate45 and matches its RTL at gate level. Next steps are replaying the traces through the RTL, longer traces, and a hardware estimate for ARC and W-TinyLFU.

References

[1] M. Naumov *et al.*, "Deep learning recommendation model for personalization and recommendation systems," arXiv:1906.00091, 2019.

[2] U. Gupta, C.-J. Wu, X. Wang, M. Naumov, B. Reagen, D. Brooks *et al.*, "The architectural implications of Facebook's DNN-based personalized recommendation," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2020.

[3] A. Eisenman, M. Naumov, D. Gardner, M. Smelyanskiy, S. Pupyrev, K. Hazelwood, A. Cidon, and S. Katti, "Bandana: Using non-volatile memory for storing deep learning models," in *Proc. Machine Learning and Systems (MLSys)*, vol. 1, 2019.

[4] M. Adnan, Y. E. Maboud, D. Mahajan, and P. J. Nair, "Accelerating recommendation system training by leveraging popular choices," *Proc. VLDB Endowment*, vol. 15, no. 1, 2022.

[5] Y. Kwon and M. Rhu, "Training personalized recommendation systems from (GPU) scratch: Look forward not backwards," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2022, pp. 860–873.

[6] L. Ke *et al.*, "RecNMP: Accelerating personalized recommendation with near-memory processing," in *Proc. ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2020.

[7] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A highly efficient cache admission policy," *ACM Trans. Storage*, vol. 13, no. 4, 2017, article 35.

[8] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[9] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2003, pp. 115–130.

[10] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[11] Criteo Labs, "Display advertising challenge," Kaggle competition, https://www.kaggle.com/competitions/criteo-display-ad-challenge, 2014.

[12] C. Wolf, "Yosys open SYnthesis suite," https://yosyshq.net/yosys/.

[13] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification (CAV)*, ser. LNCS, vol. 6174. Springer, 2010, pp. 24–40.

[14] Nangate, "FreePDK45 and the Nangate 45nm open cell library," Distributed by Si2, https://si2.org/open-cell-and-free-pdk-libraries/.

[15] Parallax Software, "OpenSTA: Parallax static timing analyzer," https://github.com/The-OpenROAD-Project/OpenSTA.

[16] S. Williams, "Icarus Verilog," https://github.com/steveicarus/iverilog.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035.

[18] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, 2011.